

Cross Validation & Hyperparameter Tuning

Contents

1	Cross Validation	2
1.1	Definition	2
1.2	k-Fold Cross Validation	2
1.3	Formula	2
1.4	Advantages	2
1.5	Disadvantages	3
1.6	Python Example	3
1.7	Common Cross Validation Techniques	3
2	Hyperparameter Tuning	4
2.1	Definition	4
2.2	Parameters vs Hyperparameters	4
2.3	Common Hyperparameters	4
2.4	Why Hyperparameter Tuning?	4
3	GridSearchCV	4
3.1	Definition	4
3.2	Example	4
3.3	Python Example	5
4	RandomizedSearchCV	5
4.1	Definition	5
4.2	Python Example	5
5	GridSearchCV vs RandomizedSearchCV	6
6	Decision Tree Hyperparameter Tuning	6
6.1	Important Parameters	6
6.2	Python Example	6
7	Interview Questions	7

1 Cross Validation

1.1 Definition

Cross Validation is a model evaluation technique used to estimate the performance of a machine learning model on unseen data. Instead of splitting the dataset only once into training and testing sets, the data is divided into multiple parts called **folds**. Each fold is used once as the testing set while the remaining folds are used for training.

Purpose:

- Evaluate model performance more reliably.
- Reduce overfitting.
- Utilize the entire dataset efficiently.
- Compare different machine learning models.

1.2 k-Fold Cross Validation

In k-Fold Cross Validation:

1. Divide the dataset into k equal folds.
2. Select one fold as the testing set.
3. Train the model on the remaining $k - 1$ folds.
4. Repeat the process until each fold has been used as the testing set.
5. Compute the average accuracy.

Example for $k = 5$

```
Fold 1 -> Test
Fold 2 -> Train
Fold 3 -> Train
Fold 4 -> Train
Fold 5 -> Train
```

Repeat until every fold becomes the test fold.

1.3 Formula

Average Accuracy

$$Accuracy = \frac{Accuracy_1 + Accuracy_2 + \dots + Accuracy_k}{k}$$

1.4 Advantages

- Better estimate of model performance.
- Reduces overfitting.

- Efficient use of data.
- Less biased evaluation.

1.5 Disadvantages

- Computationally expensive.
- Takes more training time.

1.6 Python Example

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

iris = load_iris()

X = iris.data
y = iris.target

model = DecisionTreeClassifier(random_state=42)

scores = cross_val_score(
    estimator=model,
    X=X,
    y=y,
    cv=5,
    scoring='accuracy'
)

print("Accuracy of each fold:")
print(scores)

print("Average Accuracy:")
print(scores.mean())
```

Expected Output

Accuracy of each fold:
[0.96 1.00 0.93 0.96 1.00]

Average Accuracy:
0.97

1.7 Common Cross Validation Techniques

Technique	Description
Train-Test Split	Single train-test division
K-Fold CV	Dataset divided into K equal folds
Stratified K-Fold	Maintains class distribution
Leave-One-Out CV	One sample used for testing each time
Repeated K-Fold	K-Fold repeated multiple times

2 Hyperparameter Tuning

2.1 Definition

Hyperparameters are the settings of a machine learning algorithm that are specified before training begins. They are not learned automatically by the model.

Hyperparameter Tuning is the process of finding the best combination of hyperparameter values to maximize model performance.

2.2 Parameters vs Hyperparameters

Parameters	Hyperparameters
Learned during training	Set before training
Examples: Weights, Biases	Examples: max_depth, learning_rate
Automatically updated	Manually selected or searched

2.3 Common Hyperparameters

Algorithm	Hyperparameters
Decision Tree	max_depth, criterion
Random Forest	n_estimators, max_depth
KNN	n_neighbors
SVM	C, gamma
Neural Network	epochs, batch_size, learning_rate
Logistic Regression	C

2.4 Why Hyperparameter Tuning?

- Improve model accuracy.
- Reduce overfitting.
- Reduce underfitting.
- Find the optimal model.

3 GridSearchCV

3.1 Definition

GridSearchCV exhaustively searches through every possible combination of the specified hyperparameter values using Cross Validation.

3.2 Example

Suppose

```
criterion = [gini, entropy]
```

```
max_depth = [2,3,4]
```

Possible combinations

```
(gini,2)
(gini,3)
(gini,4)
(entropy,2)
(entropy,3)
(entropy,4)
```

GridSearchCV evaluates all six combinations.

3.3 Python Example

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

iris = load_iris()

X = iris.data
y = iris.target

parameters = {
    "criterion":["gini","entropy"],
    "max_depth":[2,3,4,5]
}

grid = GridSearchCV(
    estimator=DecisionTreeClassifier(),
    param_grid=parameters,
    cv=5,
    scoring="accuracy"
)

grid.fit(X,y)

print("Best Parameters:")
print(grid.best_params_)

print("Best Accuracy:")
print(grid.best_score_)
```

4 RandomizedSearchCV

4.1 Definition

RandomizedSearchCV randomly samples a fixed number of parameter combinations rather than checking every possible combination.

It is faster than GridSearchCV, especially when the parameter search space is large.

4.2 Python Example

```
from sklearn.datasets import load_iris
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

iris = load_iris()

X = iris.data
y = iris.target

parameters = {
    "criterion":["gini","entropy"],
    "max_depth":[2,3,4,5,6],
    "min_samples_split":[2,4,6,8]
}

random = RandomizedSearchCV(
    estimator=DecisionTreeClassifier(),
    param_distributions=parameters,
    cv=5,
    n_iter=5,
    random_state=42
)

random.fit(X,y)

print(random.best_params_)
print(random.best_score_)

```

5 GridSearchCV vs RandomizedSearchCV

Feature	GridSearchCV	RandomizedSearchCV
Search Type	Exhaustive	Random
Speed	Slow	Fast
Accuracy	Higher	Good
Large Search Space	No	Yes
Computational Cost	High	Low

6 Decision Tree Hyperparameter Tuning

6.1 Important Parameters

Parameter	Description
criterion	Splitting criterion (gini or entropy)
max_depth	Maximum depth of tree
min_samples_split	Minimum samples required to split a node
min_samples_leaf	Minimum samples required in a leaf node
max_features	Number of features considered for splitting
splitter	best or random
ccp_alpha	Cost complexity pruning parameter

6.2 Python Example

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    min_samples_split=5,
    min_samples_leaf=2,
    splitter="best",
    random_state=42
)

model.fit(X,y)
```

7 Interview Questions

1. What is Cross Validation?
2. Why is k-Fold Cross Validation used?
3. What is Stratified K-Fold?
4. What are Hyperparameters?
5. Difference between Parameters and Hyperparameters?
6. What is Hyperparameter Tuning?
7. Explain GridSearchCV.
8. Explain RandomizedSearchCV.
9. Which is faster: GridSearchCV or RandomizedSearchCV?
10. Why do we use Cross Validation with GridSearchCV?